

Integer Array

INTEGER ARRAY

"Write a C program to demonstrate pointer arithmetic on an integer array. Display element values using both array indexing and pointer notation. Show address difference between consecutive elements. Sample Input: Enter number of

```
1 #include <stdio.h>
2
3 int main() {
4     int n, i;
5     int *ptr;
6
7     // Ask the user for the number of elements
8     printf("Enter number of elements: ");
9     scanf("%d", &n);
10
11     int arr[n];
12
13     printf("Enter elements:\n");
14     for (i = 0; i < n; i++) {
15         scanf("%d", &arr[i]);
16     }
17
18     ptr = arr;
19
20     printf("\nUsing Array Indexing:\n");
```

```
printf("\nUsing Array Indexing:\n");
for (i = 0; i < n; i++) {
    printf("arr[%d] = %d\n", i, arr[i]);
}

printf("\nUsing Pointer Notation:\n");
for (i = 0; i < n; i++) {
    printf("*(ptr+%d) = %d\n", i, *(ptr + i));
}

printf("\nAddress Difference:\n");
printf("Address difference between arr[0] and arr[1] = %ld bytes\n",
       (long)((char *)&arr[1] - (char *)&arr[0]));

return 0;
```

Enter number of elements: Enter elements:

Using Array Indexing:

Using Pointer Notation:

Address Difference:

Address difference between arr[0] and arr[1] = 4 bytes

Factorial

FACTORIAL

"Write a C program to find the factorial of a given number using recursion. Sample Input: Enter a number- 6 Sample Output: Factorial of 6 is 720"

```
1 #include <stdio.h>
2
3
4 long long factorial(int n) {
5
6     if (n == 0 || n == 1) {
7         return 1;
8     }
9
10    else {
11        return n * factorial(n - 1);
12    }
13 }
14
15 int main() {
16     int num;
17
18     printf("Enter a number: ");
19     scanf("%d", &num);
20
21 }
```

OUTPUT

Enter a number: Factorial of 5 is 120

```

1 #include <stdio.h>
2
3 #define N 10
4
5 void printArray(int arr[], int size) {
6     printf("[");
7     for (int i = 0; i < size; i++) {
8         printf("%d", arr[i]);
9         if (i != size - 1)
10             printf(" ");
11     }
12     printf("]\n");
13 }
14
15 void reverse(int arr[], int start, int end) {
16     while (start < end) {
17         int temp = arr[start];
18         arr[start] = arr[end];
19         arr[end] = temp;
20         start++;
21         end--;

```

OUTPUT

```

Enter 10 numbers:
Rotate Left (L)/Right (R): By positions:
Original: [1 2 3 4 5 6 7 8 9 580]
Step1: Reverse 0-7 -> [8 7 6 5 4 3 2 1 9 580]
Step2: Reverse 8-9 -> [8 7 6 5 4 3 2 1 580 9]
Step3: Reverse all -> [9 580 1 2 3 4 5 6 7 8]
Sum difference: 0

```

```

1 #include <stdio.h>
2
3 #define MAX 10
4 #define MAXNZ 100
5
6 typedef struct {
7     int r, c, v;
8 } Triple;
9
10 static int compareTriple(const void *a, const void *b) {
11     const Triple *x = (const Triple*)a;
12     const Triple *y = (const Triple*)b;
13     if (x->r != y->r) return x->r - y->r;
14     return x->c - y->c;
15 }
16
17 int main() {
18     int m, n, nz1;
19     int m2, n2, nz2;
20
21     // Read Matrix 1

```

OUTPUT

```

0 8 0
8 0 0
0 0 9

```

```

1 #include <stdio.h>
2
3 int main() {
4     int n;
5
6     printf("Enter number of terms: ");
7     scanf("%d", &n);
8
9     if (n <= 0 || n > 20) {
10         printf("Please enter a value of n between 1 and 20.\n");
11         return 0;
12     }
13
14     printf("Fibonacci series: ");
15
16     int a = 0, b = 1; // starting terms
17     for (int i = 1; i <= n; i++) {
18         if (i == 1) {
19             printf("%d", a); // 0
20         } else if (i == 2) {
21             printf(" %d", b); // 1

```

OUTPUT

```

Enter number of terms: Fibonacci series: 0 1 1 2 3

```

```

1 #include <stdio.h>
2
3 int main() {
4     int a[] = {10, 20, 30, 40, 50, 60};
5     int n = sizeof(a) / sizeof(a[0]);
6
7     int index = 4;
8     if (n < 5) {
9         printf("Array does not have 5 elements.\n");
10    } else {
11        printf("%d", a[index]);
12    }
13
14    return 0;
15 }

```

OUTPUT

50

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int cmp(const void *p, const void *q) {
5     return (*(int*)p - *(int*)q);
6 }
7
8 int main() {
9     int a[] = {10, 9, 7, 4, 3, 1, 2};
10    int n = sizeof(a) / sizeof(a[0]);
11
12    int min = a[0], max = a[0];
13    for (int i = 1; i < n; i++) {
14        if (a[i] < min) min = a[i];
15        if (a[i] > max) max = a[i];
16    }
17
18    int range = max - min + 1;
19    int *present = (int *)calloc(range, sizeof(int));
20    if (!present) return 0;
21

```

OUTPUT

5,6,8